

THỰC HÀNH VI XỬ LÝ – VI ĐIỀU KHIỂN

GVHD: Trần Ngọc Đức

Họ và tên sinh viên thực hiện: Vũ Đại Dương

Mã số sinh viên: 23520359

BÁO CÁO THỰC HÀNH 01: CỘNG 2 SỐ 32 BIT TRÊN VI XỬ LÝ 8086

I/ Chương trình cộng 2 số 32 bit.....	2
1. Khai báo tên chương trình.....	2
2. Khởi tạo các giá trị.	2
3. Xây dựng phép toán cộng	2
4. Xây dựng hàm in số nhị phân 16 bit ra màn hình.....	3
5. In giá trị tổng ra màn hình.	4
II/ Lưu đồ giải thuật xử lý chương trình cộng 2 số 32 bit.....	5
III/ Chương trình trừ 2 số 32 bit	6
IV/ Giải thích chương trình đã thực hiện	6
1. Sử dụng stack.....	6
2. Thêm chữ vào màn hình console	7
V/ Tài liệu tham khảo	8

I/ Chương trình cộng 2 số 32 bit

1. Khai báo tên chương trình.

name "add-sub-32bit"

2. Khởi tạo các giá trị.

- Vì phần mềm emu8086 không có sẵn các số 32 bit, nên ta sẽ khởi tạo 2 số 16 bit thay cho 1 số 32 bit. Vậy nên, thay vì khởi tạo 2 số 32 bit, ta sẽ khởi tạo 4 số 16 bit lần lượt là ax, bx, cx, dx với (bx-dx) là số 32 bit đầu tiên, (ax-cx) là số 32 bit thứ 2

- Đoạn lệnh khởi tạo:

mov bx, 0001h

mov dx, 0001h

mov ax, 0001h

mov cx, 0FFFFh

- Dựa vào đoạn khởi tạo các giá trị này, ta sẽ biết số 32 bit đầu tiên là 0001 0001₁₆ và số 32 bit thứ hai là 0001 FFFF₁₆. Với biến cx, phải thêm 0 trước giá trị để chương trình có thể hiểu được.

3. Xây dựng phép toán cộng

bx – dx

+ ax - cx

- Đầu tiên, ta dùng lệnh “clc” để xóa giá trị của cờ tràn (CF) để tránh xảy ra lỗi khi thực hiện phép cộng.

- Thực hiện phép cộng từ phải sang trái, điều đó cũng đồng nghĩa với việc thực hiện (dx + cx) trước, ta sử dụng lệnh “add dx, cx”. Sau đó, nếu có tràn, ta sẽ cộng giá trị tràn này với (bx + ax), ta sử dụng lệnh “adc bx, ax”.

- Vì trong khi thực hiện lệnh in giá trị bx ra màn hình, dx sẽ bị thay đổi nên ta sẽ lưu giá trị dx vào di bằng lệnh “mov di, dx”

- Đoạn lệnh thực hiện phép cộng:

clc

add dx, cx

adc bx, ax

mov di, dx

4. Xây dựng hàm in số nhị phân 16 bit ra màn hình

*Đoạn lệnh in 1 ký tự ra màn hình:

```
mov dl, 'b'
```

```
mov ah, 2
```

```
int 21h
```

=> In “b” ra màn hình

- Cho số cần in được gán trong biến si.

- Đoạn lệnh thực hiện in số nhị phân 16 bit:

```
print_bin:
```

```
    mov cx, 16
```

```
print_loop:
```

```
    mov ah, 2
```

```
    mov dl, '0'
```

```
    test si, 8000h
```

```
    jz zero
```

```
    mov dl, '1'
```

```
zero:
```

```
    int 21h
```

```
    shl si, 1
```

```
loop print_loop
```

```
    ret
```

- Gán cx = 16 (in 16 bit nhị phân ra màn hình – tương tự vòng lặp i trong C)

- Lệnh test (như phép logic AND) để kiểm tra bit đầu tiên của số, nếu là 0 thì in “0”, là 1 thì in “1”

- Sau đó, nhảy đến ký tự sau ký tự đã xét, lặp lại hàm để tiếp tục thực hiện việc in.

- Sau khi hoàn thành, dùng lệnh “ret” để kết thúc hàm.

5. In giá trị tổng ra màn hình.

- Sau khi hoàn thành phép cộng, ta sẽ biết bx là 16 bit đầu tiên của tổng, dx là 16 bit cuối của tổng
- Ta sẽ in 16 bit trong biến bx trước → Gán si = bx → Gọi hàm print_bin để thực hiện việc in 16 bit của bx
- Sau đó in <space> để dễ kiểm tra lỗi sai hơn
- Gán si = dx → Gọi hàm print_bin để thực hiện việc in 16 bit của dx
- Cuối cùng in ra ký tự “b” để biết kết quả là binary

- Đoạn lệnh thực hiện in tổng 32 bit ra màn hình:

```
mov si, bx
```

```
call print_bin
```

```
mov dl, ''
```

```
mov ah, 2
```

```
int 21h
```

```
mov si, di
```

```
call print_bin
```

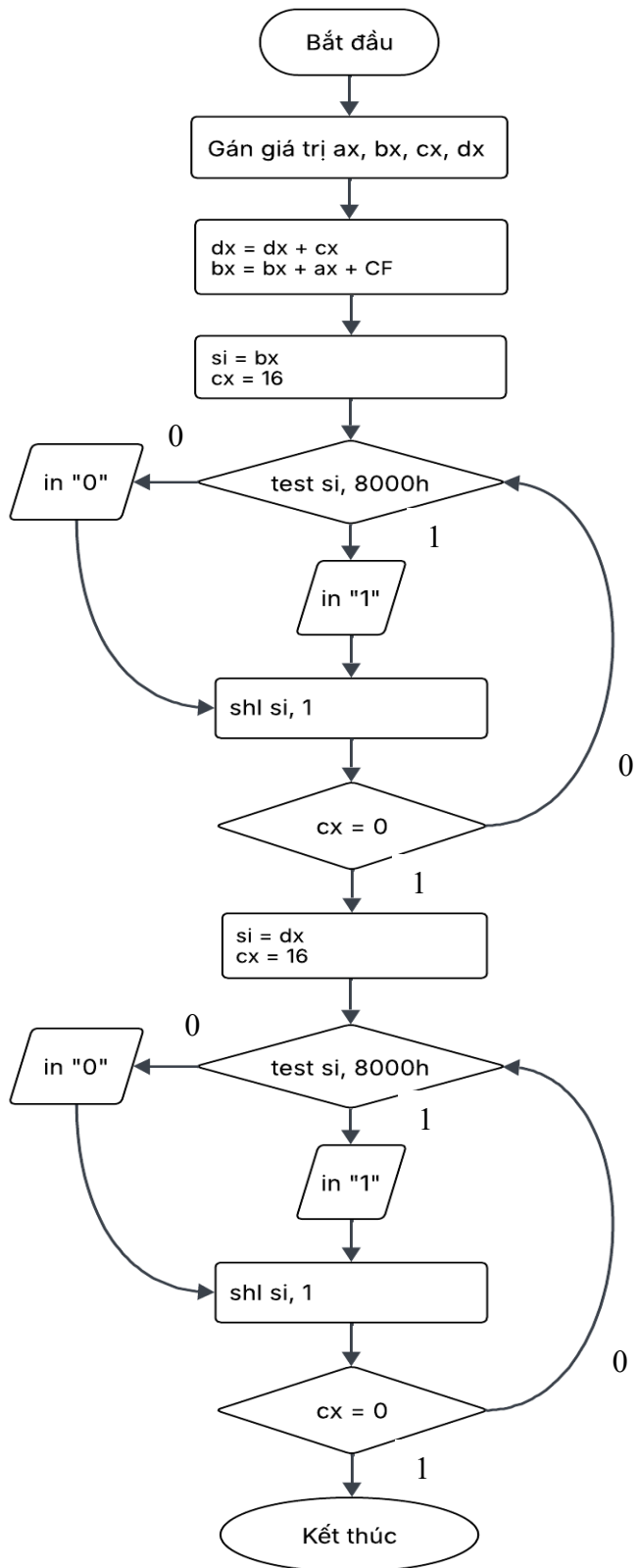
```
mov dl, 'b'
```

```
mov ah, 2
```

```
int 21h
```

- Để kết thúc chương trình, dùng lệnh “ret”.

II/ Lưu đồ giải thuật xử lý chương trình cộng 2 số 32 bit



III/ Chương trình trừ 2 số 32 bit

- Chương trình trừ 2 số 32 bit giống như chương trình cộng 2 số 32 bit, chỉ khác ở việc xây dựng phép trừ.

- Đoạn lệnh thực hiện phép trừ:

clc

sub dx, cx

sbb bx, ax

mov di, dx

- Phép trừ thực hiện từ phải sang trái, nghĩa là thực hiện dx – cx trước, sau đó thực hiện bx – ax – CF (cờ tràn).

- Dùng di để lưu lại giá trị dx.

IV/ Giải thích chương trình đã thực hiện

- Thay vì chia ra làm 2 chương trình khác nhau, em đã gộp 2 chương trình lại thành 1 chương trình có chức năng thực hiện cả cộng và trừ 2 số 32 bit

- Quy trình làm: thực hiện phép cộng, sau đó thực hiện phép trừ

1. Sử dụng stack

- Vì sau khi thực hiện phép cộng, các biến sẽ thay đổi giá trị → sau khi khai báo biến, em đã đẩy giá trị các biến vào stack bằng lệnh “push” → sau khi hoàn thành thực hiện phép cộng, em lấy các giá trị trong stack ra và gán vào từng biến bằng lệnh “pop”.

- Vì stack thực hiện theo giải thuật LIFO (Last In – First Out) nên e đã thực hiện như sau:

...

mov bx, 0001h

mov dx, 0001h

mov ax, 0001h

mov cx, 0FFFFh

push ax

push bx

push cx

push dx

...

pop dx

pop cx

pop bx

pop ax

...

- Sau khi đã lấy các giá trị ra khỏi stack và gán vào các biến, em tiếp tục thực hiện phép trừ.

2. Thêm chữ vào màn hình console

- Để dễ dàng phân biệt tổng hiệu, em đã thực hiện in “Tong: “ và “Hieu: “ ra màn hình bằng lệnh:

+ In “Tong: “

mov dx, offset msg_tong

mov ah, 09h

int 21h

msg_tong db 'Tong: \$'

+ In “Hieu: “

mov dx, offset msg_hieu

mov ah, 09h

int 21h

msg_hieu db 'Hieu: \$'

- Cùng với đó là in dấu Enter đã dễ nhìn hơn bằng lệnh:

mov dl, 0Ah

mov ah, 02h

int 21h

mov dl, 0Dh

int 21h

V/ Tài liệu tham khảo

- Kết quả chạy của chương trình:

<https://drive.google.com/file/d/168aSg7IXLgEXeLu2vqix00KQmYpQr2XE/view?usp=sharing>